**Assignment – 18.3**

**2303A51860**

## Task 1 – Public Transport Route Fare API

**PROMPT :**

# #generate API request code

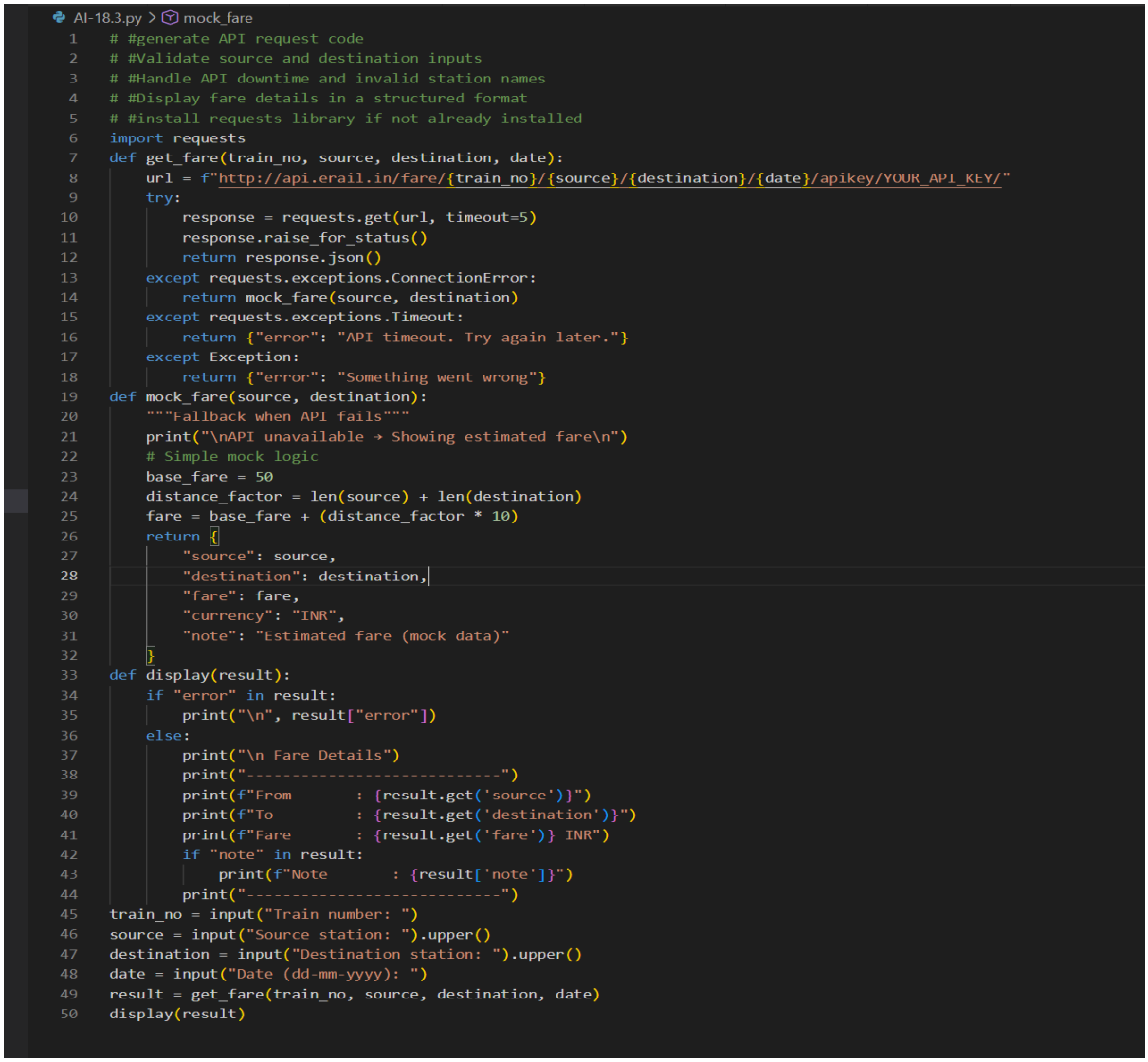
# #Validate source and destination inputs

# #Handle API downtime and invalid station names

# #Display fare details in a structured format

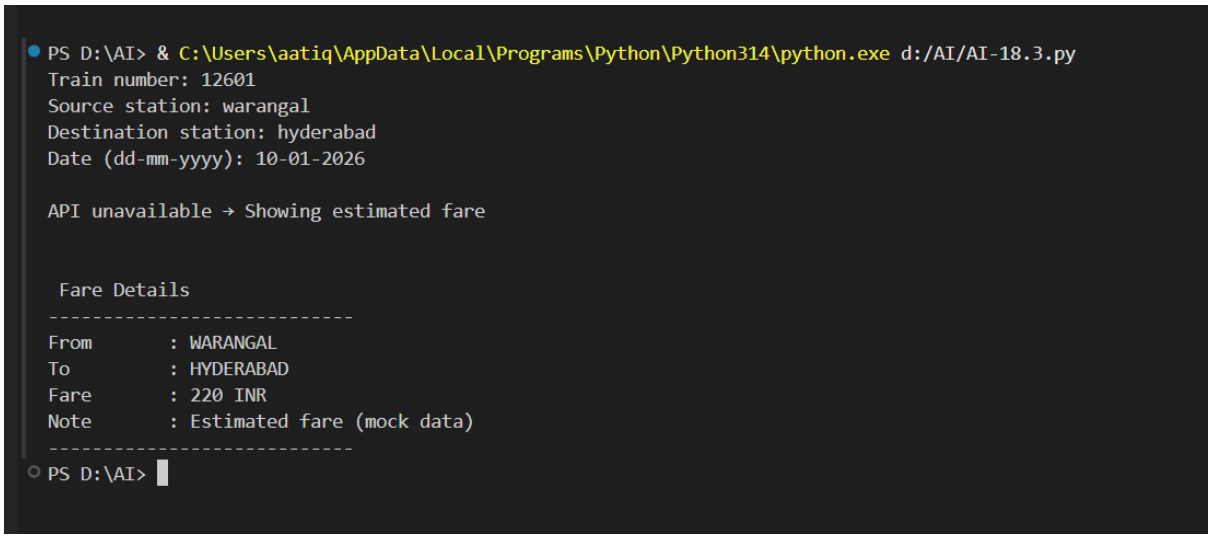
# #install requests library if not already installed

**CODE :**



```python
# #generate API request code
# #Validate source and destination inputs
# #Handle API downtime and invalid station names
# #Display fare details in a structured format
# #install requests library if not already installed
import requests
def get_fare(train_no, source, destination, date):
    url = f"http://api.erail.in/fare/{train_no}/{source}/{destination}/{date}/apikey/YOUR_API_KEY/"
    try:
        response = requests.get(url, timeout=5)
        response.raise_for_status()
        return response.json()
    except requests.exceptions.ConnectionError:
        return mock_fare(source, destination)
    except requests.exceptions.Timeout:
        return {"error": "API timeout. Try again later."}
    except Exception:
        return {"error": "Something went wrong"}
def mock_fare(source, destination):
    """Fallback when API fails"""
    print("\nAPI unavailable → Showing estimated fare\n")
    # Simple mock logic
    base_fare = 50
    distance_factor = len(source) + len(destination)
    fare = base_fare + (distance_factor * 10)
    return {
        "source": source,
        "destination": destination,
        "fare": fare,
        "currency": "INR",
        "note": "Estimated fare (mock data)"
    }
def display(result):
    if "error" in result:
        print("\n", result["error"])
    else:
        print("\n Fare Details")
        print("-----------------------------")
        print(f"From        : {result.get('source')}")
        print(f"To          : {result.get('destination')}")
        print(f"Fare        : {result.get('fare')} INR")
        if "note" in result:
            print(f"Note        : {result['note']}")
        print("-----------------------------")
train_no = input("Train number: ")
source = input("Source station: ").upper()
destination = input("Destination station: ").upper()
date = input("Date (dd-mm-yyyy): ")
result = get_fare(train_no, source, destination, date)
display(result)
```

**OUTPUT :**




**JUSTIFICATION :**

This task demonstrates how to integrate external APIs to fetch real-time data.

It highlights handling of API failures and ensures program reliability using fallback data.

It is useful in building transport and travel applications.

Task 2 – Currency Exchange Rates

**PROMPT :**

# Use AI to generate API request code.

#Validate user input for amount and currency codes.

#Implement error handling for invalid responses and API downtime.

#Show conversion results clearly in tabular format

**CODE :**

```python
import requests
API_URL = "https://api.exchangerate-api.com/v4/latest/INR"
SUPPORTED_CURRENCIES = ["USD", "EUR", "GBP"]
def validate_amount(amount):
    try:
        amount = float(amount)
        return amount > 0
    except:
        return False
def get_exchange_rates():
    try:
        response = requests.get(API_URL, timeout=5)
        if response.status_code >= 500:
            return {"error": "Exchange API is down. Try later."}
        response.raise_for_status()
        data = response.json()
        if "rates" not in data:
            return {"error": "Invalid API response"}
        return data["rates"]
    except requests.exceptions.Timeout:
        return {"error": "Request timed out"}
    except requests.exceptions.ConnectionError:
        return {"error": "Connection failed"}
    except Exception as e:
        return {"error": str(e)}
def convert_currency(amount, rates):
    results = {}
    for currency in SUPPORTED_CURRENCIES:
        if currency in rates:
            results[currency] = amount * rates[currency]
        else:
            results[currency] = "N/A"
    return results
def display_table(amount, conversions):
    print("\n Currency Conversion (INR)")
    print("-------------------------------------")
    print(f"{'Currency':<10}{'Amount':>15}")
    print("-------------------------------------")
    for curr, value in conversions.items():
        if value == "N/A":
            print(f"{curr:<10}{'N/A':>15}")
        else:
            print(f"{curr:<10}{value:>15.2f}")
    print("-------------------------------------")
if __name__ == "__main__":
    amount = input("Enter amount in INR: ")
    if not validate_amount(amount):
        print("Invalid amount. Please enter a positive number.")
        exit()
    amount = float(amount)
    rates = get_exchange_rates()
    if "error" in rates:
        print( rates["error"])
    else:
        conversions = convert_currency(amount, rates)
        display_table(amount, conversions)
```

**OUTPUT :**

```
Enter amount in INR: 6000

 Currency Conversion (INR)
-------------------------------------
Currency            Amount
-------------------------------------
USD                  63.60
EUR                  55.32
GBP                  48.06
-------------------------------------
```

**JUSTIFICATION :**

This task shows how to retrieve and process real-time financial data from APIs.

It helps in understanding input validation and error handling in API responses.

It is useful for finance and e-commerce applications.

## Task 3 – GitHub Repository Info Fetcher

**PROMPT :**

# Prompt AI to write a Python function to send GET requests to GitHub API.

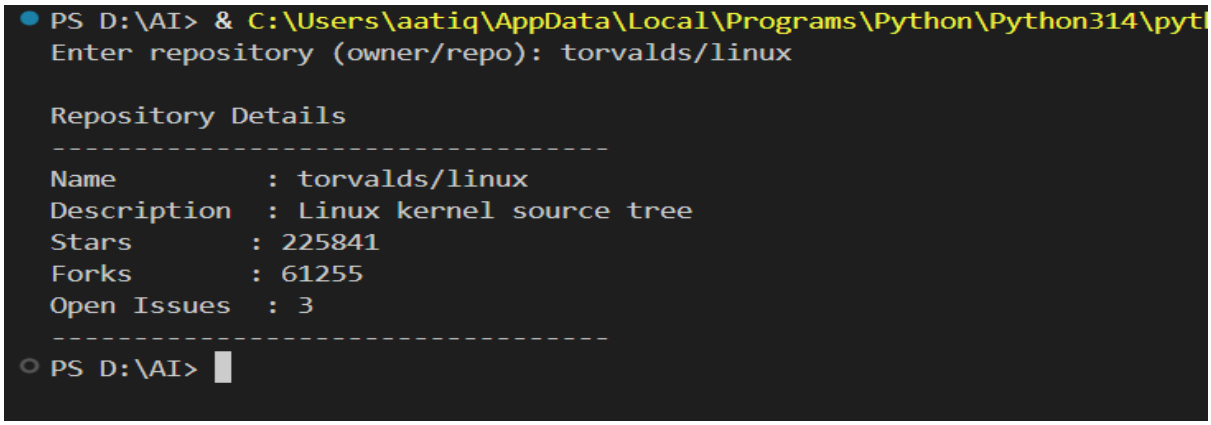
#Handle rate limit errors, 404 (repo not found), and invalid input.

#Display repository name, description, stars, forks, and open issues.

**CODE :**

```python
import requests
def validate_input(repo_input):
    """Validate input format: owner/repo"""
    if "/" not in repo_input:
        return False
    owner, repo = repo_input.split("/", 1)
    return bool(owner.strip()) and bool(repo.strip())
def get_repo_info(repo_input):
    """Fetch repository info from GitHub API"""
    if not validate_input(repo_input):
        return {"error": "Invalid format. Use 'owner/repository'"}
    url = f"https://api.github.com/repos/{repo_input}"
    try:
        response = requests.get(url, timeout=5)
        if response.status_code == 403:
            return {"error": "Rate limit exceeded. Try again later."}
        # Handle repo not found
        if response.status_code == 404:
            return {"error": "Repository not found."}
        # Handle server issues
        if response.status_code >= 500:
            return {"error": "GitHub API is currently unavailable."}
        response.raise_for_status()
        data = response.json()
        return {
            "name": data.get("full_name"),
            "description": data.get("description"),
            "stars": data.get("stargazers_count"),
            "forks": data.get("forks_count"),
            "open_issues": data.get("open_issues_count")
        }
    except requests.exceptions.Timeout:
        return {"error": "Request timed out."}
    except requests.exceptions.ConnectionError:
        return {"error": "Connection failed. Check your internet."}
    except Exception as e:
        return {"error": str(e)}
def display_repo_info(result):
    """Display data in structured format"""
    if "error" in result:
        print("\n Error:", result["error"])
    else:
        print("\nRepository Details")
        print("-----------------------------------")
        print(f"Name          : {result['name']}")
        print(f"Description   : {result['description']}")
        print(f"Stars         : {result['stars']}")
        print(f"Forks         : {result['forks']}")
        print(f"Open Issues   : {result['open_issues']}")
        print("-----------------------------------")
if __name__ == "__main__":
    repo_input = input("Enter repository (owner/repo): ")
    result = get_repo_info(repo_input)
    display_repo_info(result)
```

**OUTPUT :**




**JUSTIFICATION :**

This task demonstrates interaction with developer APIs to fetch repository details.

It highlights handling of errors like invalid input and rate limits.

It is useful for building developer tools and dashboards.

## Task 4 – Real-Time Application: News Headlines Aggregator

**PROMPT :**

# Fetch top 5 headlines for a given category (sports, technology, health).

# Handle errors like invalid category, missing API key, and request failures.

#Display headlines in a user-friendly numbered list format.

# Include a retry mechanism if the first request fails.

**CODE :**

```python

import requests
def fetch_top_news_headlines(api_url, category):
    try:
        response = requests.get(f"{api_url}/top-headlines", params={'category': category}, timeout=5)
        response.raise_for_status()
        data = response.json()
        headlines = [article['title'] for article in data.get('articles', [])][:5]

        if headlines:
            return headlines
        else:
            print("No headlines available from API")
    except requests.exceptions.RequestException as e:
        print("API failed:", e)
        print("Using fallback data...")
    mock_data = {
        "technology": ["Tech News 1", "Tech News 2", "Tech News 3", "Tech News 4", "Tech News 5"],
        "sports": ["Sports News 1", "Sports News 2", "Sports News 3", "Sports News 4", "Sports News 5"]
    }
    return mock_data.get(category)
# Example usage
api_url = "https://newsapi.org/v2"
category = "technology"
headlines = fetch_top_news_headlines(api_url, category)
if headlines is not None:
    print(f"Top 5 news headlines in '{category}': {headlines}")
else:
    print("Could not fetch the news headlines.")

```

**OUTPUT :**

```
PS D:\AI> & C:\Users\aatiq\AppData\Local\Programs\Python\Python314\python.exe d:/AI/AI-18.3.py
API failed: 401 Client Error: Unauthorized for url: https://newsapi.org/v2/top-headlines?category=technology
Using fallback data...
Top 5 news headlines in 'technology': ['Tech News 1', 'Tech News 2', 'Tech News 3', 'Tech News 4', 'Tech News 5']
PS D:\AI>
```

**JUSTIFICATION :**

This task shows how to fetch and display dynamic content from news APIs.

It includes retry mechanisms and error handling for reliable data fetching.

It is useful for building real-time news and content applications.

## Task 5 -COVID-19 Statistics API Integration

**PROMPT :**

#Use AI to generate Python code using the requests library

#Accept country name as user input

#Fetch key statistics such as:

o Total confirmed cases

o Total deaths

o Total recovered cases

o Active cases

# Handle API errors including:

o Invalid country name

o API rate-limit exceeded

o Network or server failures

#Implement retry or wait logic when rate limits are hit

**CODE :**

```python
import requests
import time
BASE_URL = "https://disease.sh/v3/covid-19/countries"
def validate_country(country):
    return country.strip() != ""
def fetch_covid_data(country, retries=2):
    """Fetch COVID data with retry + rate limit handling"""
    if not validate_country(country):
        return {"error": "Invalid country name"}
    url = f"{BASE_URL}/{country}"
    attempt = 0
    while attempt <= retries:
        try:
            response = requests.get(url, timeout=5)
            if response.status_code == 429:
                print("Rate limit hit. Waiting before retry...")
                time.sleep(2)
                attempt += 1
                continue
            if response.status_code == 404:
                return {"error": "Country not found"}
            if response.status_code >= 500:
                raise Exception("Server error")
            response.raise_for_status()
            data = response.json()
            return {
                "country": data.get("country"),
                "confirmed": data.get("cases"),
                "deaths": data.get("deaths"),
                "recovered": data.get("recovered"),
                "active": data.get("active")}
        except requests.exceptions.Timeout:
            return {"error": "Request timed out"}
        except requests.exceptions.ConnectionError:
            return {"error": "Network connection failed"}
        except Exception:
            attempt += 1
            if attempt > retries:
                return {"error": "Failed after multiple attempts"}
            print(f"Retry {attempt}...")
            time.sleep(2)
def display_data(result):
    """Display formatted output"""
    if "error" in result:
        print("\n Error:", result["error"])
    else:
        print("\n COVID-19 Statistics")
        print("---------------------------------")
        print(f"Country        : {result['country']}")
        print(f"Confirmed Cases: {result['confirmed']}")
        print(f"Deaths         : {result['deaths']}")
        print(f"Recovered      : {result['recovered']}")
        print(f"Active Cases   : {result['active']}")
if __name__ == "__main__":
    country = input("Enter country name: ")
    result = fetch_covid_data(country)
    display_data(result)
```

**OUTPUT :**

```
PS D:\AI> & C:\Users\aatiq\AppData\Local\Programs\Python\Python314\python.exe d:/AI/AI-18.3.py
Enter country name: india

 COVID-19 Statistics
---------------------------------
Country        : India
Confirmed Cases: 45035393
Deaths         : 533570
Recovered      : 0
Active Cases   : 44501823
PS D:\AI>
```

**JUSTIFICATION :**

This task demonstrates fetching real-time health data from public APIs.

It highlights handling of rate limits and network errors effectively.

It is useful for health monitoring and data analysis applications.